
Chapter 4

One-dimensional array

1. The Array type

A variable of array type is no longer a simple variable, but a complex data structure, grouping information of the same type, accessible by indexes indicating their location. An array can be seen as a group of variables of the same type with the same name.

We distinguish two types of arrays: one-dimensional arrays (Vectors) and multi-dimensional arrays (Matrices).

Dimension: The dimension of an array is the number of indices needed to identify a single element.

Index: When data is stored in an array, the element is identified by an index which is a non-negative integer (≥ 0). The index ranges from 0 to $N - 1$ (for C language) or from 1 to N (where N is the array size).

Array element: Elements are items stored in an array and can be accessed by their index.

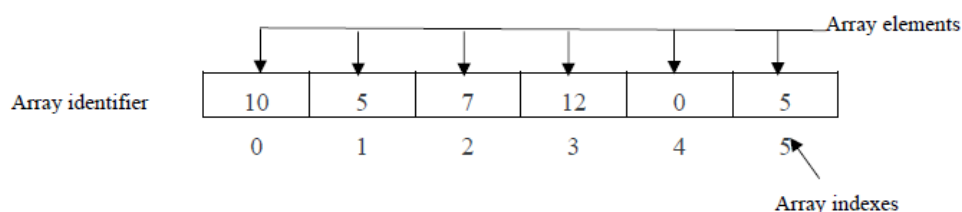
Array Length: The length of an array is determined by the number of elements it can contain.

2. One-Dimensional Array

It is also referred to as a vector, allowing you to access any of its elements using a single index. Each index value selects a specific element from the array.

2.1. Representation

The array is stored in memory as a sequence of contiguous cells. Once created, new cells cannot be added to or removed from the array (static). The figure below illustrates an array consisting of 8 real numbers.



2.2. Declaration

Var array_name : array [size] of elementType;

- arrayName: The identifier name given to the array (the variable name).
- Size: The number of elements in the array.
- elementType: The type of elements in the array, which can be of any type like integer (int), float, ...

To declare multiple arrays of the same type, use a comma "," while specifying the size of each array between square brackets [] or ().

Example

const N=100

Var

marks [N] : **array of** real

tab1[50],tab2[20] : **array of** integer;

2.3. Accessing Array Elements

An element from an array is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array and the expression inside the brackets evaluates to an integer value.

Syntax to refer to a specific element in the array:

array_name[index] ;

Example

Var Tab : array [8] of integer;

	index	0	1	2	3	4	5	6	7
Tab=	value	15	7	2-	0	9	2	0	3-

To access the first element of array 'tab', we use tab[0].

To access the fourth element, we use tab[3].

tab[5-3]←tab[tab[3]+1] ⇔ tab[2]←tab[0+1] ⇔ tab[2]←7

The array Tab becomes

Indice	0	1	2	3	4	5	6	7
valeur	15	7	7	0	9	2	0	-3

Note: Accessing an element that doesn't exist (if the index is greater than the array size or negative) will cause the program to terminate.

2.4. Assignment

Assigning a value x to an element i of a numeric array T is done by:

$T[i] \leftarrow vx$

For example, the statement: $T[0] \leftarrow 5$; assigns the value 5 to the first element of the array T .

2.5. Reading an Array

To fill an array of N numbers, we use "read" N times like:

read (tab[0])

read (tab[1])

...

read (tab[N-1])

However, it's observed that the read instruction is repeated N times, iterating from 0 to $N-1$.

Hence, the "for" loop can be employed with the counter serving as the index.

for ($I \leftarrow 0$ to $N-1$) do

read(tab[i])

endFor

or the reverse order

for ($i \leftarrow N-1$ to 0

read(tab[i])

endFor

Example : Initializing the elements to 0

$i = 0$

while ($i < N-1$ Do)

$T[i] \leftarrow 0$; $i \leftarrow i+1$; endWhile

The "read" instruction is used for populating the table, while the "write" instruction is employed to display to the user what is needed.

2.6. Displaying an Array

We substitute the read operation with the write operation.

write (tab[0])

write (tab[1])

...

write (tab[N-1])

```
for (i ← 0 to N-1) do
print(tab[i])
endFor
```

```
or the reverse order
for (i← N-1) to 0
print(T[i])
endFor
```

Note

- To visit all elements of an array, we generally use the "**for**" loop.
- The array size must be specified during programming (declaration), but we can give the user the impression that the array size can be changed by declaring a large array and using only part of it. We ask the user for the desired size, it must not exceed the actual array size.

2.7. Example

Write an algorithm that receives the averages of N students, where N is determined by the user, then calculates the number of students who failed the subject (average less than 10).

```
Algorithm nb_adjourned
var avg :array(100) of real;
i, aj, N : integer
Begin
print ("enter number of students (<", MAX, ")")
read (N)
for (i ← 0 to N-1) do
read (avg[i]);
endfor
aj ← 0
for i ← 0 to N-1 do
  if (avg[i]<10) then
aj ← aj+1;
end if
end for
print("the number of adjourned is ", aj)
End.
```

2.8. Array algorithms

2.8.1. Searching for elements in an array

Linear search is a straightforward algorithm that traverses the array, comparing each element to the target element. If the target element is found, a Boolean variable is set to true; otherwise, it remains false. Let's consider an array named `tab` with a length of `N`, and an element `x` that we aim to search for:

```
found ← false ; i ← 0 ;
while ((found = false) and (i < N)) do
if (tab[i] = x) then
found ← true ;
endIf ;
i ← i + 1 ;
endWhile
```

2.8.2. Insertion of an element in an array

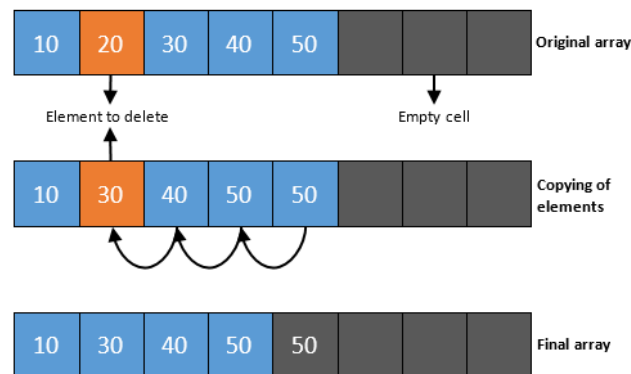
The algorithm which inserts a given value in unsorted array.

```
// Input the value to be inserted and its position
print("Input the value to be inserted : ");
read(x);
printf("Input the Position, where the value to be inserted :");
read(p);
// Move all data to the right side of the array to make space
for (i ← n to p)
    tab[i] = tab[i - 1];
endFor
// Insert the new value at the given position
tab[p - 1] = x;
// Display the array after insertion
print("After Insert the element the new array is :");
for (i ← 0 to n)
    print(tab[i]);
endFor
```

2.8.3. Deleting an element in an array

Logic to remove element from array

1. Move to the specified location which you want to remove in given array.
2. Copy the next element to the current element of array. Which is you need to perform $\text{array}[i] = \text{array}[i + 1]$.
3. Repeat above steps till last element of array.
4. Finally decrement the size of array by one.



// Algorithm to delete an element from array at specified position

```
Algorithm deleting;
Const MAX_SIZE 100;
var
  arr: array [MAX_SIZE] of integer;
  i, size, pos : integer;
Begin
  /* Input size and element in array */
  print("Enter size of the array : ");
  read(size);
  print("Enter elements in array : ");
  for(i=0 to size) do
    read("%d", &arr[i]);
  endFor
  /* Input element position to delete */
  print("Enter the element position to delete : ");
  read(pos);
```

```

/* Invalid delete position */
if(pos < 0 or pos > size) then
    print("Invalid position! Please enter position between 1 to size);
else
    /* Copy next element value to current element */
    for(i=pos-1 to i<size-1) do
        arr[i] = arr[i + 1];
    endFor
    /* Decrement array size by 1 */
    size= size-1-;
    /* Print array after deletion */
    print("Elements of array after delete are : ");
    for(i=0 to size) do
        printf("%d\t", arr[i]);
    endFor
End.

```

2.8.4. Sorting Arrays

Sorting is one of the basic operations on the arrays. Indeed; it's usually helpful when we have an array to be able to put it in some sort of order (be it numerical or alphabetical).

There are many algorithms to sort a list. Some are simple and some are complex. Some are fast while others are slow. We'll look at a few of the most common ones.

Selection sort

Selection sort is a simple sorting algorithm that works by repeatedly finding the smallest element in an unsorted array and swapping it with the first element in the array. This process is repeated until the entire array is sorted.

```

for (i ← 0 to (N - 2) do
    smallest = i ;
    for j ← (i + 1) to N-1 do
        if (tab[j] < tab[smallest] )then
            smallest = j;
        endif
    endFor ;
    temp = tab[i]; tab[i] = tab[j]; tab[j] = temp;
endFor

```

Bubble Sort

Bubble sort is a simple sorting algorithm that works by repeatedly comparing adjacent elements in an array and swapping them if they are in the wrong order. Thus, the smallest element "bubbles" to the top of the array. This process is repeated until the entire array is sorted.

```
for (i ← 0 to n-2 ) do
  for (j ← 0 to n-i-1) do
    if (tab[j]>tab[j+1]) then
      tmp ← tab[j];
      tab[j] ← tab[j+1];
      tab[j+1] ← tmp;
    endif;
  endFor ;
endFor.
```